

Tribhuvan University
Madan Bhandari Memorial College
New Baneshwor, Kathmandu



Lab report on
“OOP in Java”

Submitted by

Name: Roj Dahal

Code no: 3279624

Faculty: Humanities and Social Science

Semester: Third Semester

Submitted to

Mr. Hemant Yadav

Table of content

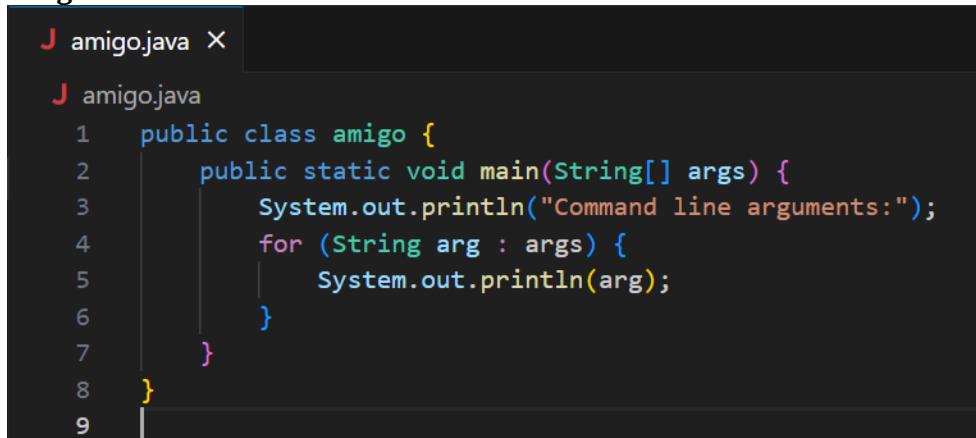
SN	Index	Signature
1	Writing simple programs to demonstrate command line arguments	
2	Writing programs that include variables, data types, and operators	
3	Writing programs that include all control statements	
4	Writing programs with arrays	
5	Writing programs with classes and inheritance	
6	Writing programs with concepts of packages	
7	Writing programs with exception handling	
8	Writing programs to handle string methods	
9	Writing multithreaded programs using Thread and Runnable	
10	Writing programs with console and file input/output	
11	Writing programs using java.lang and java.util classes and interfaces	
12	Writing programs using collection classes	
13	Writing GUI programs using swing components, event handling, and layout management	
14	Writing Java applets	
15	Writing programs using JDBC to handle databases	

Lab-1

Writing Simple Programs to Demonstrate Command Line Arguments

Objective: Understand how to handle and utilize command line arguments in Java programs to dynamically input data at runtime.

Program:

A screenshot of an IDE window titled 'amigo.java'. The code is as follows:

```
1 public class amigo {
2     public static void main(String[] args) {
3         System.out.println("Command line arguments:");
4         for (String arg : args) {
5             System.out.println(arg);
6         }
7     }
8 }
9
```

Output

A screenshot of the IDE's output window. It shows the command used to run the program and the resulting output.

```
PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Running] cd "c:\Users\lenovo\Desktop\jav\" && javac amigo.java && java amigo
Command line arguments:

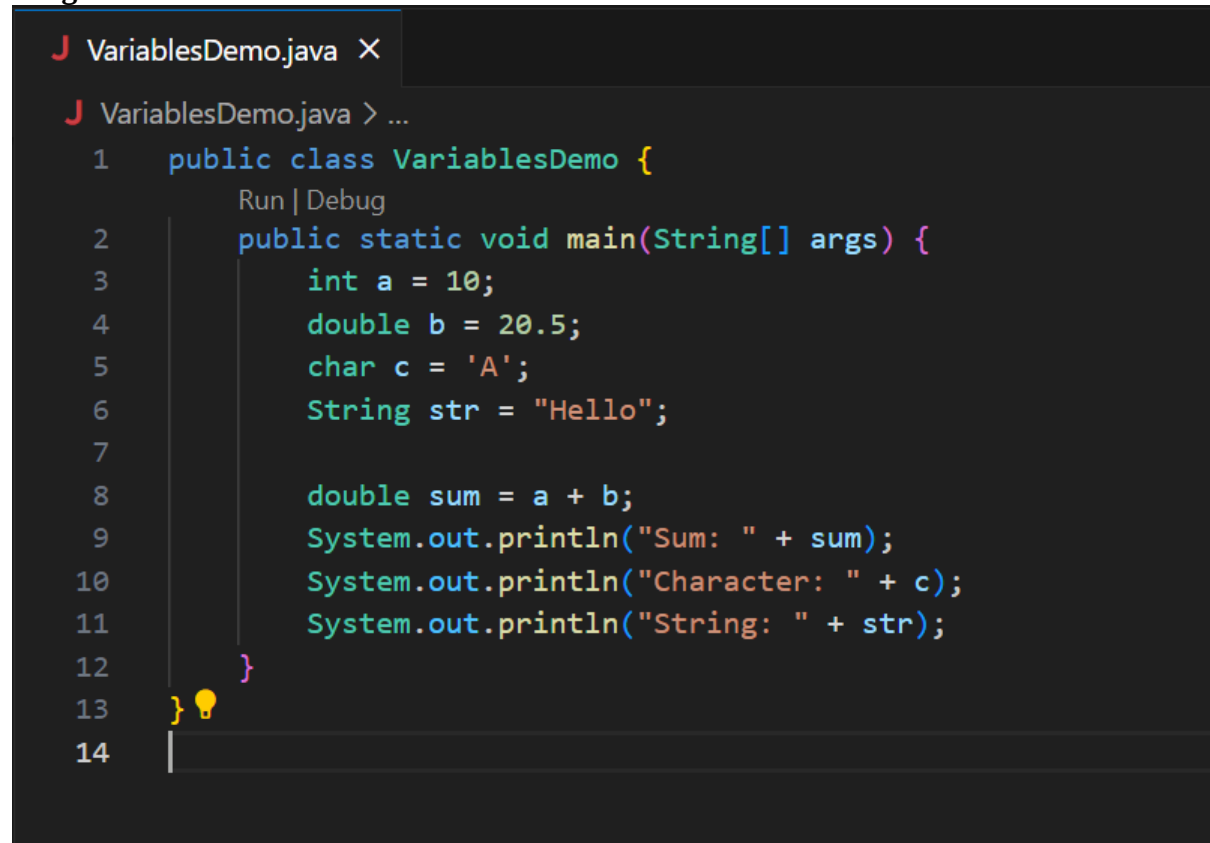
[Done] exited with code=0 in 1.299 seconds
```

Lab-2

Writing Programs that Include Variables, Data Types, and Operators

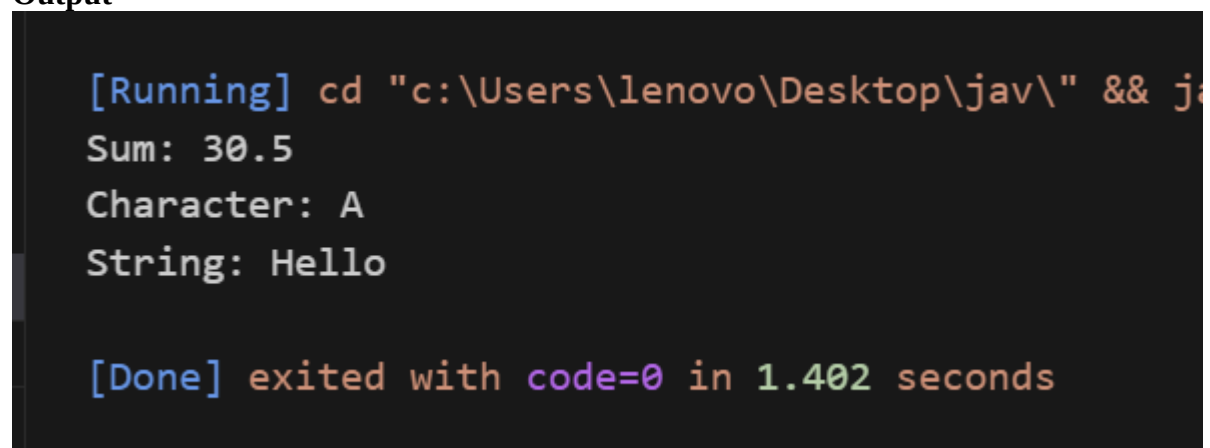
Objective: Learn the basics of Java variables, data types, and operators to perform simple computations and data manipulations.

Program:



```
J VariablesDemo.java X
J VariablesDemo.java > ...
1  public class VariablesDemo {
    Run | Debug
2      public static void main(String[] args) {
3          int a = 10;
4          double b = 20.5;
5          char c = 'A';
6          String str = "Hello";
7
8          double sum = a + b;
9          System.out.println("Sum: " + sum);
10         System.out.println("Character: " + c);
11         System.out.println("String: " + str);
12     }
13 }
14
```

Output



```
[Running] cd "c:\Users\lenovo\Desktop\jav\" && java VariablesDemo
Sum: 30.5
Character: A
String: Hello

[Done] exited with code=0 in 1.402 seconds
```

Lab-3

Writing Programs that Include All Control Statements

Objective: Master the use of Java control statements (if-else, switch, for loop, while loop, do-while loop) to control the flow of program execution based on conditions.

Program

```
J ControlStatementsDemo.java •
J ControlStatementsDemo.java > ControlStatementsDemo > main(String[])
1 public class ControlStatementsDemo {
2     public static void main(String[] args) {
3         int num = 5;
4         if (num > 0) {
5             System.out.println(x:"Positive number");
6         } else {
7             System.out.println(x:"Non-positive number");
8         }
9         switch (num) {
10            case 1:
11                System.out.println(x:"Number is one");
12                break;
13            case 5:
14                System.out.println(x:"Number is five");
15                break;
16            default:
17                System.out.println(x:"Number is neither one nor five");
18        }
19        for (int i = 0; i < 5; i++) {
20            System.out.println("For loop iteration: " + i);
21        }
22        int i = 0;
23        while (i < 5) {
24            System.out.println("While loop iteration: " + i);
25            i++;
26        }
27        i = 0;
28        do {
29            System.out.println("Do-while loop iteration: " + i);
30            i++;
31        } while (i < 5);
32    }
33 }
34
```

Output

```
PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL
[Running] cd "c:\Users\lenovo\Desktop\jav\" && j
Positive number
Number is five
For loop iteration: 0
For loop iteration: 1
For loop iteration: 2
For loop iteration: 3
For loop iteration: 4
While loop iteration: 0
While loop iteration: 1
While loop iteration: 2
While loop iteration: 3
While loop iteration: 4
Do-while loop iteration: 0
Do-while loop iteration: 1
Do-while loop iteration: 2
Do-while loop iteration: 3
Do-while loop iteration: 4
[Done] exited with code=0 in 1.433 seconds
```

Lab-4

Writing Programs with Arrays

Objective: Understand how to declare, initialize, and manipulate arrays in Java for storing and processing collections of data.

Program

```
J ArraysDemo.java X
J ArraysDemo.java > ...
1  public class ArraysDemo {
    Run | Debug
2      public static void main(String[] args) {
3          int[] numbers = {1, 2, 3, 4, 5};
4
5          for (int num : numbers) {
6              System.out.println("Number: " + num);
7          }
8      }
9  }
10
```

Output

```
[Running] cd "c:\Users\lenovo\D
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

[Done] exited with code=0 in 1.
```

Lab-5

Writing Programs with Classes and Inheritance

Objective: Learn the principles of object oriented programming by creating classes and implementing inheritance to promote code reusability and organization.

Program

```
J InheritanceDemo.java X
J InheritanceDemo.java > InheritanceDemo
1  class animal {
2      void eat() {
3          System.out.println(x:"This animal eats food.");
4      }
5  }
6
7  class Dog extends animal {
8      void bark() {
9          System.out.println(x:"The dog barks.");
10     }
11 }
12
13 public class InheritanceDemo {
14     Run | Debug
15     public static void main(String[] args) {
16         Dog dog = new Dog();
17         dog.eat();
18         dog.bark();
19     }
```

Output

```
[Done] exited with code=0 in 1.207 seconds

[Running] cd "c:\Users\lenovo\Desktop\jav\"
This animal eats food.
The dog barks.

[Done] exited with code=0 in 1.341 seconds
```

Lab-6

Writing Programs with Concepts of Packages

Objective: Understand the organization of Java classes into packages for better modularity and namespace management, and learn how to import and use packages.

Program

```
MyPackageClass.java X
MyPackageClass.java > ...
1 // MyPackageClass.java
2 package mypackage;
3
4 public class MyPackageClass {
5     public void display() {
6         System.out.println("Hello from the package!");
7     }
8 }
9
```

```
PackageDemo.java X
PackageDemo.java > ...
1 // PackageDemo.java
2 import mypackage.MyPackageClass;
3
4 public class PackageDemo {
5     Run | Debug
6     public static void main(String[] args) {
7         MyPackageClass obj = new MyPackageClass();
8         obj.display();
9     }
10 }
```

Output

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PC
[Running] cd "c:\Users\lenovo\Desktop\jav\" && j
Hello from the package!

[Done] exited with code=0 in 1.298 seconds
```

Lab-7

Writing Programs with Exception Handling

Objective: Learn to handle runtime errors gracefully using Java exception handling mechanisms (try, catch, finally) to create robust programs.

Program

```
ExceptionHandlingDemo.java X
ExceptionHandlingDemo.java > ...
1  public class ExceptionHandlingDemo {
    Run | Debug
2  public static void main(String[] args) {
3      try {
4          int a = 10;
5          int b = 0;
6          int result = a / b; // This line will throw an ArithmeticException
7          System.out.println("Result: " + result); // Use the variable to avoid the warning
8      } catch (ArithmeticException e) {
9          System.out.println(x:"Exception caught: Division by zero!");
10     } finally {
11         System.out.println(x:"Finally block executed.");
12     }
13 }
14 }
15
```

Output

```
[Running] cd "c:\Users\lenovo\Desktop\jav\" && javac E
Exception caught: Division by zero!
Finally block executed.

[Done] exited with code=0 in 1.312 seconds
```

Lab-8

Writing Programs to Handle String Methods

Objective: Explore the various methods provided by the Java String class to perform common operations on strings, such as manipulation, comparison, and conversion.

Program

```
StringMethodsDemo.java X
StringMethodsDemo.java > ...
1 public class StringMethodsDemo {
    Run | Debug
2     public static void main(String[] args) {
3         String str = "Hello World";
4
5         System.out.println("Length: " + str.length());
6         System.out.println("Character at index 4: " + str.charAt(index:4));
7         System.out.println("Substring from index 6: " + str.substring(beginIndex:6));
8         System.out.println("Uppercase: " + str.toUpperCase());
9         System.out.println("Lowercase: " + str.toLowerCase());
10        System.out.println("Contains 'World': " + str.contains(s:"World"));
11    }
12 }
13
```

Output

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL P
[Running] cd "c:\Users\lenovo\Desktop\jav\" &&
Length: 11
Character at index 4: o
Substring from index 6: World
Uppercase: HELLO WORLD
Lowercase: hello world
Contains 'World': true

[Done] exited with code=0 in 1.418 seconds
```

Lab-9

Writing Multithreaded Programs using Thread and Runnable

Objective: Understand the concepts of multithreading in Java and how to create and manage threads using the Thread class and Runnable interface to perform concurrent tasks.

Program

```
J MultithreadingDemo.java X
J MultithreadingDemo.java > ...
1  class MyThread extends Thread {
2      public void run() {
3          for (int i = 0; i < 5; i++) {
4              System.out.println("Thread: " + i);
5          }
6      }
7  }
8
9  class MyRunnable implements Runnable {
10     public void run() {
11         for (int i = 0; i < 5; i++) {
12             System.out.println("Runnable: " + i);
13         }
14     }
15 }
16
```

Output

```
15 }
16
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PO
[Running] cd c:\users\lenovo\desktop\jav\ && J
Runnable: 0
Runnable: 1
Runnable: 2
Runnable: 3
Runnable: 4
Thread: 0
Thread: 1
Thread: 2
Thread: 3
Thread: 4
```

Lab-10

Writing Programs with Console and File Input/Output

Objective: Learn how to perform input and output operations using the console and file system in Java to read from and write to files.

Program

```
J FileIODemo.java X
J FileIODemo.java > FileIODemo > main(String[])
1  import java.io.*;
2
3  public class FileIODemo {
4      Run | Debug
5      // Write to a file
6      try (FileWriter writer = new FileWriter(fileName:"output.txt")) {
7          writer.write(str:"Hello, file!");
8      } catch (IOException e) {
9          e.printStackTrace();
10     }
11
12     // Read from a file
13     try (BufferedReader reader = new BufferedReader(new FileReader(fileName:"output.txt"))) {
14         String line;
15         while ((line = reader.readLine()) != null) {
16             System.out.println("Read from file: " + line);
17         }
18     } catch (IOException e) {
19         e.printStackTrace();
20     }
21 }
22 }
```

Output

```
[Done] exited with code=0 in 1.545 seconds

[Running] cd "c:\Users\lenovo\Desktop\jav\" && javac Fil
Read from file: Hello, file!

[Done] exited with code=0 in 1.494 seconds
```

Lab-11

Writing Programs using java.lang and java.util Classes and Interfaces

Objective: Familiarize with essential classes and interfaces from the java.lang and java.util packages to leverage built-in Java functionality for common tasks such as date handling and collection management.

Program

```
UtilClassesDemo.java X
UtilClassesDemo.java > ...
1  import java.util.Date;
2  import java.util.ArrayList;
3
4  public class UtilClassesDemo {
5      Run | Debug
6      public static void main(String[] args) {
7          // Using java.util.Date
8          Date date = new Date();
9          System.out.println("Current date: " + date);
10
11         // Using java.util.ArrayList
12         ArrayList<String> list = new ArrayList<>();
13         list.add(e:"Apple");
14         list.add(e:"Banana");
15         list.add(e:"Cherry");
16
17         for (String fruit : list) {
18             System.out.println("Fruit: " + fruit);
19         }
20     }
21 }
```

Output

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Done] exited with code=0 in 1.641 seconds

[Running] cd "c:\Users\lenovo\Desktop\jav\" && java
Current date: Fri Jun 21 17:57:49 NPT 2024
Fruit: Apple
Fruit: Banana
Fruit: Cherry

[Done] exited with code=0 in 1.535 seconds
```

Lab-12

Writing Programs using Collection Classes

Objective: Explore the Java Collections Framework to store, retrieve, and manipulate groups of objects using various collection classes such as ArrayList, HashMap, and HashSet.

Program

```
J CollectionsDemo.java X
J CollectionsDemo.java > ...
3
4 public class CollectionsDemo {
    Run | Debug
5     public static void main(String[] args) {
6         // Using HashMap
7         Map<String, Integer> map = new HashMap<>();
8         map.put(key:"Apple", value:1);
9         map.put(key:"Banana", value:2);
10        map.put(key:"Cherry", value:3);
11
12        for (Map.Entry<String, Integer> entry : map.entrySet()) {
13            System.out.println(entry.getKey() + ": " + entry.getValue());
14        }
15    }
16 }
17
```

Output

```
[Done] exited with code=1 in 1.216 seconds

[Running] cd "c:\Users\lenovo\Desktop\jav\" &
Apple: 1
Cherry: 3
Banana: 2

[Done] exited with code=0 in 1.515 seconds
```

Lab-13

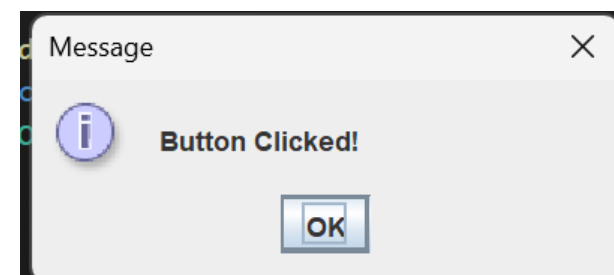
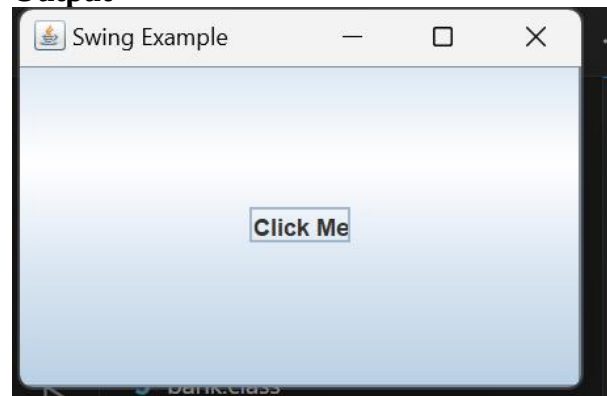
Writing GUI Programs using Swing Components, Event Handling, and Layout Management

Objective: Learn to create graphical user interfaces in Java using Swing components, handle user interactions through event listeners, and manage the layout of components within the GUI.

Program

```
SwingDemo.java X
SwingDemo.java > ...
1  import javax.swing.*;
2  import java.awt.event.ActionEvent;
3  import java.awt.event.ActionListener;
4
5  public class SwingDemo {
6      public static void main(String[] args) {
7          JFrame frame = new JFrame(title:"Swing Example");
8          JButton button = new JButton(text:"Click Me");
9
10         button.addActionListener(new ActionListener() {
11             public void actionPerformed(ActionEvent e) {
12                 JOptionPane.showMessageDialog(parentComponent:null, message:"Button Clicked!");
13             }
14         });
15
16         frame.add(button);
17         frame.setSize(width:300, height:200);
18         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19         frame.setVisible(b:true);
20     }
21 }
22
```

Output

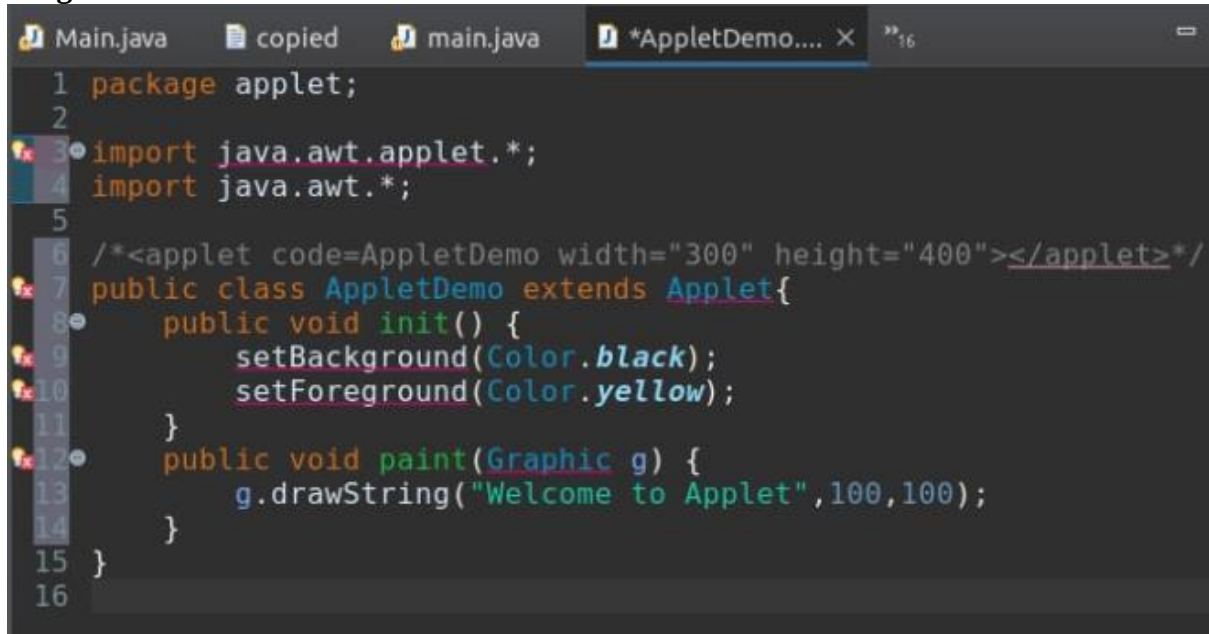


Lab-14

Writing Java Applets

Objective: Understand the basics of creating and running Java applets, a type of Java program that can be embedded in a web page and run in a web browser.

Program

A screenshot of an IDE window with a dark theme. The title bar shows 'Main.java', 'copied', 'main.java', and '*AppletDemo.... X'. The code is as follows:

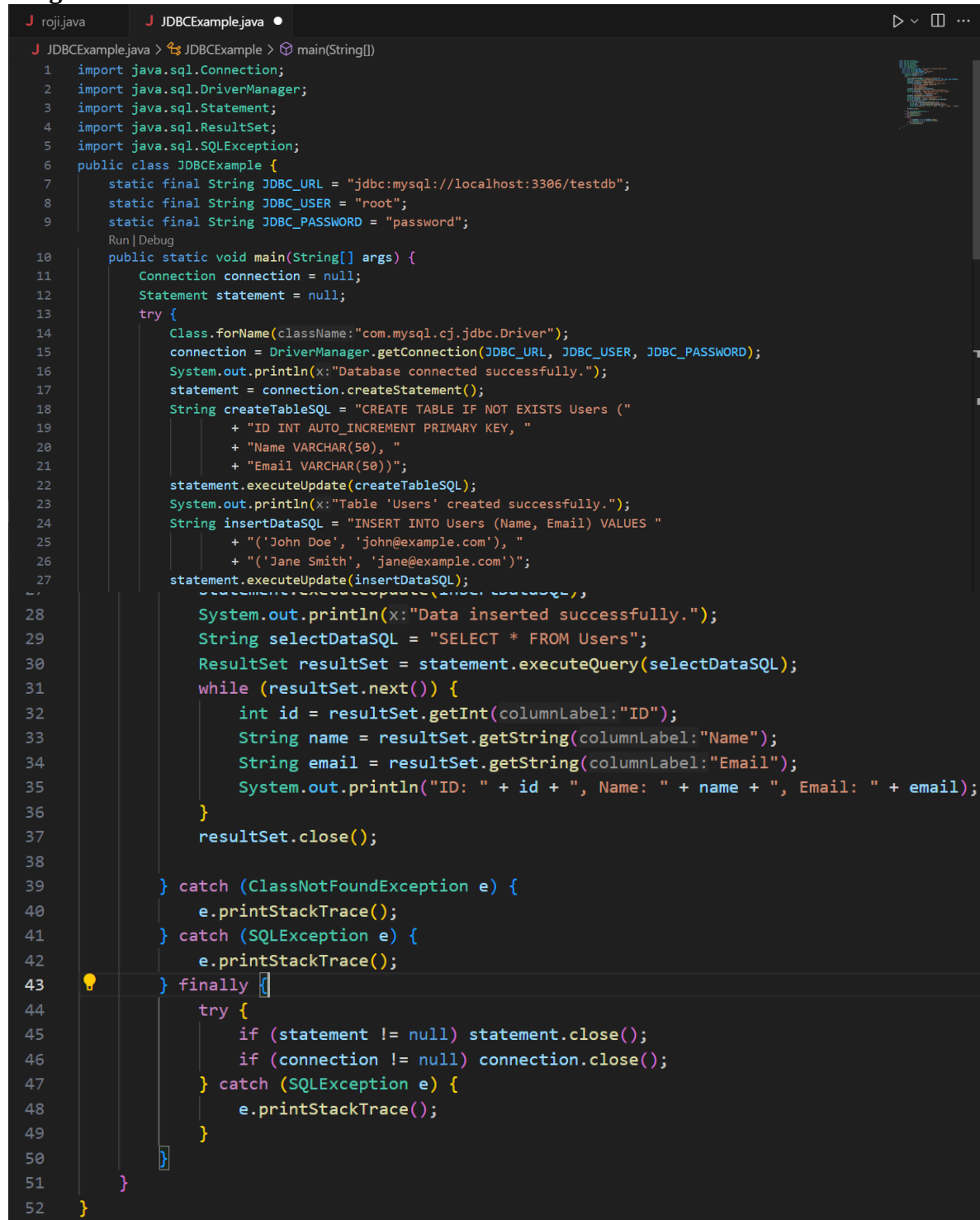
```
1 package applet;
2
3 import java.awt.applet.*;
4 import java.awt.*;
5
6 /*<applet code=AppletDemo width="300" height="400"></applet>*/
7 public class AppletDemo extends Applet{
8     public void init() {
9         setBackground(Color.black);
10        setForeground(Color.yellow);
11    }
12    public void paint(Graphics g) {
13        g.drawString("Welcome to Applet",100,100);
14    }
15 }
16
```

Lab-15

Writing Programs using JDBC to Handle Databases

Objective: Learn to interact with databases using Java Database Connectivity (JDBC) to perform database operations such as querying, updating, and managing data within a relational database.

Program



```
J roji.java J JDBCExample.java •
J JDBCExample.java > JDBCExample > main(String[])
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.Statement;
4 import java.sql.ResultSet;
5 import java.sql.SQLException;
6 public class JDBCExample {
7     static final String JDBC_URL = "jdbc:mysql://localhost:3306/testdb";
8     static final String JDBC_USER = "root";
9     static final String JDBC_PASSWORD = "password";
10
11     public static void main(String[] args) {
12         Connection connection = null;
13         Statement statement = null;
14         try {
15             Class.forName(className: "com.mysql.cj.jdbc.Driver");
16             connection = DriverManager.getConnection(JDBC_URL, JDBC_USER, JDBC_PASSWORD);
17             System.out.println(x: "Database connected successfully.");
18             statement = connection.createStatement();
19             String createTableSQL = "CREATE TABLE IF NOT EXISTS Users ("
20                 + "ID INT AUTO_INCREMENT PRIMARY KEY, "
21                 + "Name VARCHAR(50), "
22                 + "Email VARCHAR(50))";
23             statement.executeUpdate(createTableSQL);
24             System.out.println(x: "Table 'Users' created successfully.");
25             String insertDataSQL = "INSERT INTO Users (Name, Email) VALUES "
26                 + "('John Doe', 'john@example.com'), "
27                 + "('Jane Smith', 'jane@example.com')";
28             statement.executeUpdate(insertDataSQL);
29             System.out.println(x: "Data inserted successfully.");
30             String selectDataSQL = "SELECT * FROM Users";
31             ResultSet resultSet = statement.executeQuery(selectDataSQL);
32             while (resultSet.next()) {
33                 int id = resultSet.getInt(columnLabel: "ID");
34                 String name = resultSet.getString(columnLabel: "Name");
35                 String email = resultSet.getString(columnLabel: "Email");
36                 System.out.println("ID: " + id + ", Name: " + name + ", Email: " + email);
37             }
38             resultSet.close();
39         } catch (ClassNotFoundException e) {
40             e.printStackTrace();
41         } catch (SQLException e) {
42             e.printStackTrace();
43         } finally {
44             try {
45                 if (statement != null) statement.close();
46                 if (connection != null) connection.close();
47             } catch (SQLException e) {
48                 e.printStackTrace();
49             }
50         }
51     }
52 }
```